Chapter 12 Data Structures

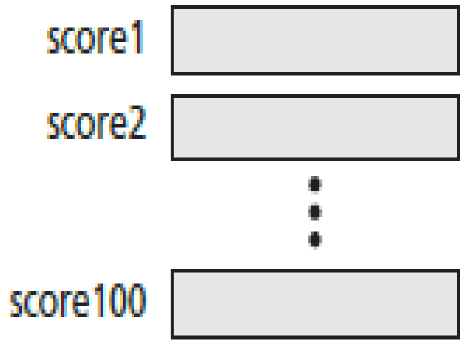
After studying this chapter, the student should be able to:

- Define a data structure.
- Define an array as a data structure and how it is used to store a list of data.
- Distinguish between the name of an array and the names of the elements in an array.
- Describe operations defined for an array.
- Define a record as a data structure and how it is used to store attributes.
- Distinguish between the name of a record and the names of its fields.
- Define a linked list as a data structure and how it is implemented using pointers.
- Describe operations defined for a linked list.
- Compare and contrast arrays, records, and linked lists.
- Define the applications of arrays, records, and linked lists.

12.1 ARRAYS

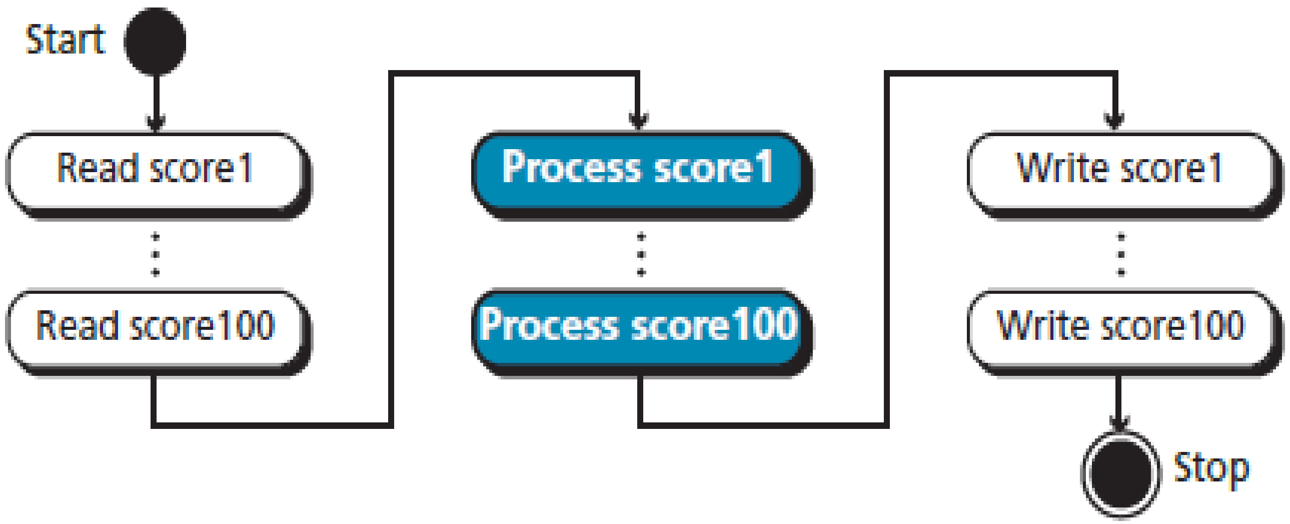
- Imagine that we have 100 scores. We need to read them, process them, and print them. We must also keep these 100 scores in memory for the duration of the program.
- We can define a hundred variables, each with a different name, as shown in Figure 12.1.

Figure 12.1 *A hundred individual variables*



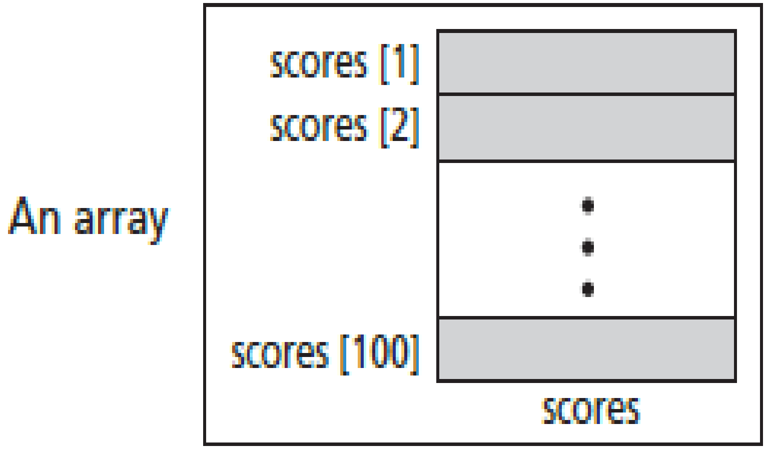
- But having 100 different names creates other problems. We need 100 references to read them, 100 references to process them, and 100 references to write them.
- Figure 12.2 shows a diagram that illustrates this problem.

Figure 12.2 *Processing individual variables*



- An array is a sequenced collection of elements, normally of the same data type, although some programming languages accept arrays in which elements are of different types. We can refer to the elements in the array as the first element, the second element, and so forth until we get to the last element.

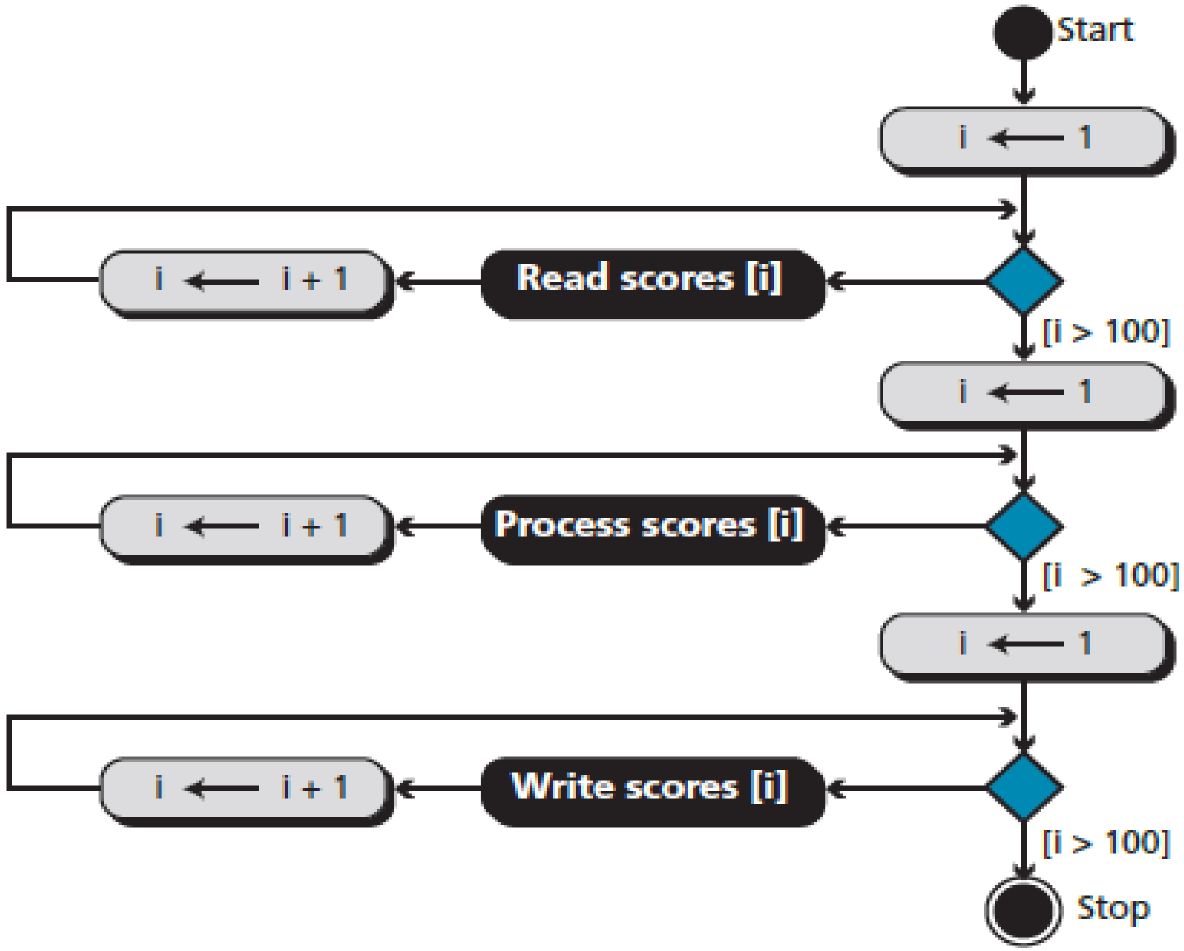
Figure 12.3 *Arrays with indexes*



- We can use loops to read and write the elements in an array. We can also use loops to process elements. Now it does not matter if there are 100, 1000, or 10,000 elements to be processed—loops make it easy to handle them all. We can use an integer variable to control the loop, and remain in the loop as long as the value of this variable is less than the total number of elements in the array (Figure 12.4).

We have used indexes that start from some modern languages such as C, C++, and Java start indexes from 0.

Figure 12.4 *Processing an array*



Example 12.1

- Compare the number of instructions needed to handle 100 individual elements in Figure 11.2 and the array with 100 in Figure 11.4. Assume that processing each score needs only one instruction.

Solution

- In the first case, we need 100 instructions to read, 100 instructions to write, and 100 instructions to process. The total is 300 instructions.
- In the second case, we have three loops. In each loop we have two instructions, for a total of six instructions. However, we also need three instructions for initializing the index and three instruction to check the value of the index. In total, we have twelve instructions.

Example 12.2

- In computer science, one of the big issues is the reusability of programs—for example, how much needs to be changed if the number of data items is changed. Assume we have written two programs to process the scores as shown in Figure 11.2 and Figure 11.4. If the number of scores changes from 100 to 1000, how many changes do we need to make in each program?

Solution

- In the first program we need to add $3 \times 900 = 2700$ instructions. In the second program, we only need to change three conditions ($I > 100$ to $I > 1000$).
- We can actually modify the diagram in Figure 11.4 to reduce the number of changes to one.

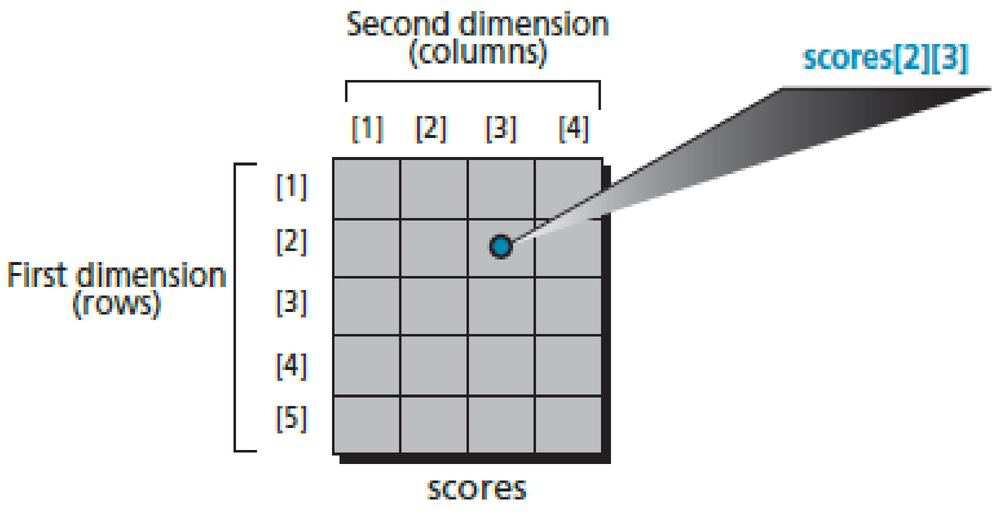
12.1.2 Array name versus element name

- In an array we have two types of identifiers: the name of the array and the name of each individual element. The name of the array is the name of the whole structure, while the name of an element allows us to refer to that element. In the array of Figure 12.3, the name of the array is ***scores*** and name of each element is the name of the array followed by the index, for example, ***scores*[1]**, ***scores*[2]**, and so on. In this chapter, we mostly need the names of the elements, but in some languages, such as C, we also need to use the name of the array.

12.1.2 Multi-dimensional arrays

- The arrays discussed so far are known as **one-dimensional** arrays because the data is organized linearly in only one direction. Many applications require that data be stored in more than one dimension. Figure 12.5 shows a table, which is commonly called a **two-dimensional** array.

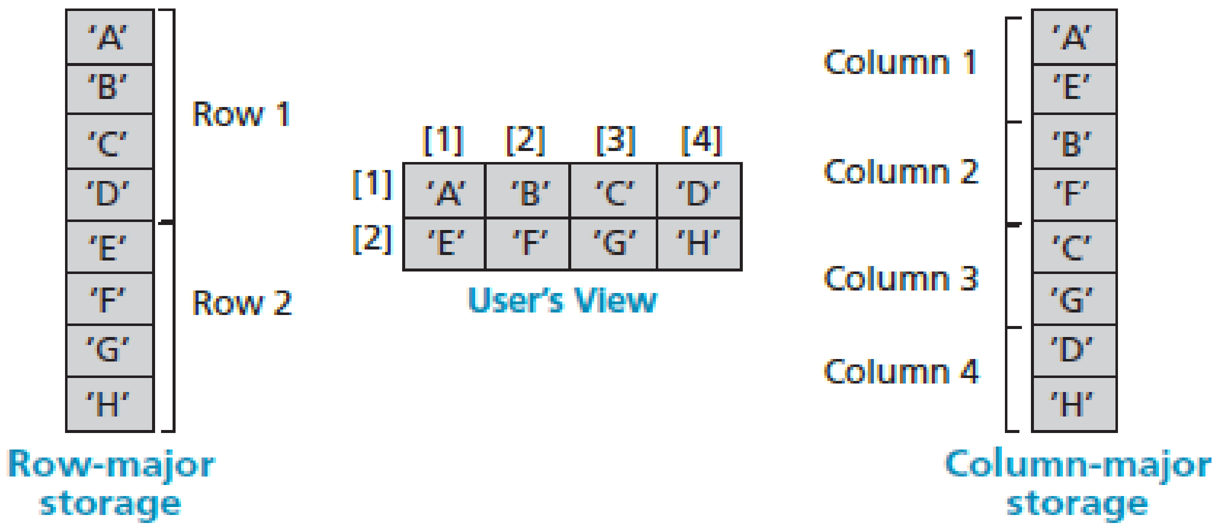
Figure 12.5 *A two-dimensional array*



12.1.3 Memory layout

- The indexes in a one-dimensional array directly define the relative positions of the element in actual memory. Figure 12.6 shows a two-dimensional array and how it is stored in memory using row-major or column-major storage. Row-major storage is more common.

Figure 12.6 *Memory layout of arrays*



Example 12.3

- We have stored the two-dimensional array students in memory. The array is 100 × 4 (100 rows and 4 columns). Show the address of the element students[5][3] assuming that the element student[1][1] is stored in the memory location with address 1000 and each element occupies only one memory location. The computer uses row-major storage.

Solution

- We can use the following formula to find the location of an element, assuming each element occupies one memory location.

$$y = x + \text{Cols} \times (i - 1) + (j - 1)$$

- If the first element occupies the location 1000, the target element occupies the location

$$y = x + \text{Cols} \times (i - 1) + (j - 1) = 1000 + 4(5 - 1) + (3 - 1) = 1018$$

12.1.4 Operations on array

- Although we can apply conventional operations defined for each element of an array (see Chapter 4), there are some operations that we can define on an array as a data structure. The common operations on arrays as structures are **searching**, **insertion**, **deletion**, **retrieval**, and **traversal**.
- Although searching, retrieval, and traversal of an array is an easy job, insertion and deletion is time consuming. The elements need to be shifted down before insertion and shifted up after deletion.

- Algorithm 12.1 gives an example of finding the average of elements in array whose elements are reals.

Algorithm 12.1 Calculating the average of elements in an array

```
Algorithm: ArrayAverage (Array, n)
Purpose: Find the average value
Pre: Given the array Array and the number of elements, n.
Post: None
Return: The average value
{
    sum ← 0.0
    i ← 1
    while (i ≤ n)
    {
        sum ← sum + Array [i]
        i ← i + 1
    }
    average ← sum / n
    Return ( average )
}
```

12.1.5 Strings

- A string as a set of characters is treated differently in different languages.
- In C, a string is an array of characters.
- In C++, a string can be an array of characters, but there is a type name string. In Java, a string is a type.

12.1.6 Application

- Thinking about the operations discussed in the previous section gives a clue to the application of arrays. If we have a list in which a lot of insertions and deletions are expected after the original list has been created, we should not use an array.
- An array is more suitable when the number of deletions and insertions is small, but a lot of searching and retrieval activities are expected.

12.2 RECORDS

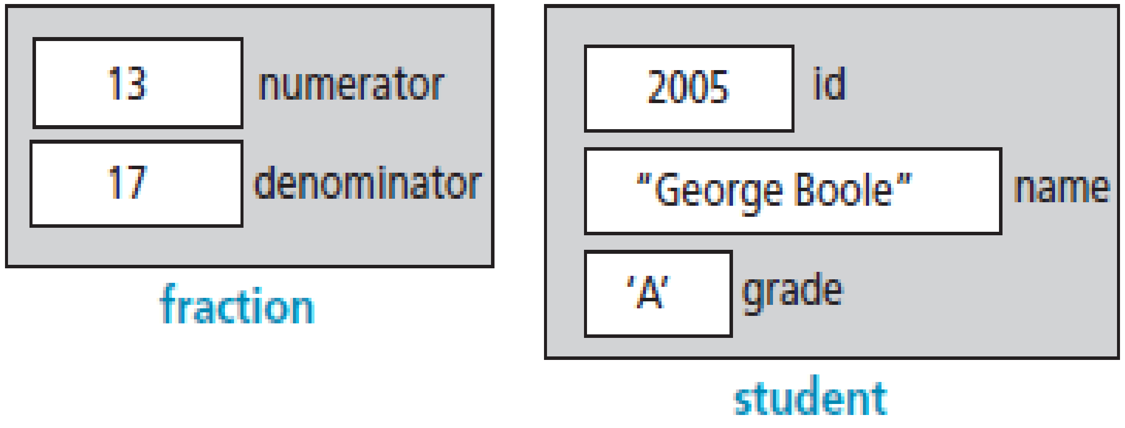
- A record is a collection of related elements, possibly of different types, having a single name.
- Each element in a record is called a field.
- A field is the smallest element of named data that has meaning.
- A field has a type, and exists in memory.
- Fields can be assigned values, which in turn can be accessed for selection or manipulation.
- A field differs from a variable primarily in that it is part of a record.

The elements in a record can be of the same or different types, but all elements in the record must be related.

Figure 12.7 contains two examples of records.

- The first example, fraction, has two fields, both of which are integers.
- The second example, student, has three fields made up of three different types.

Figure 12.7 *Records*



12.2.1 Record name *versus* field name

- Just like in an array, we have two types of identifier in a record: the name of the record and the name of each individual field inside the record. The name of the record is the name of the whole structure, while the name of each field allows us to refer to that field.
- For example, in the student record of Figure 12.7, the name of the record is student, the name of the fields are **student.id**, **student.name**, and **student.grade**. Most programming languages use a ***period*** (.) to separate the name of the structure (record) from the name of its components (fields). This is the convention we use in this book.

Example 12.4

The following shows how the value of fields in Figure 12.7 are stored:

student.id ← 2005   student.name ← "G. Boole"   student.grade ← 'A'

12.2.2 Comparison of records and arrays

- We can conceptually compare an array with a record. This helps us to understand when we should use an array and when a record. An array defines a combination of elements, while a record defines the identifiable parts of an element. For example, an array can define a class of students (40 students), but a record defines different attributes of a student, such as id, name, or grade.

Example 12.5

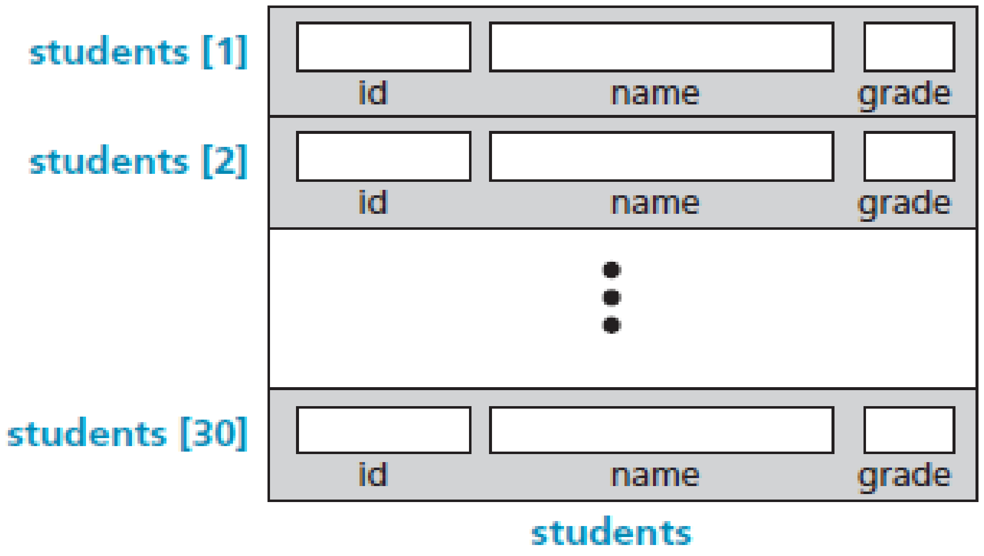
The following shows how we access the fields of each record in the students array to store values in them.

(students[1]).id ← 1001	(students[1]).name ← "J. Aron"	(students[1]).grade ← 'A'
(students[2]).id ← 2007	(students[2]).name ← "F. Bush"	(students[2]).grade ← 'F'
...	...	...
(students[30]).id ← 3012	(students[30]).name ← "M. Blair"	(students[1]).grade ← 'B'

Array of records

- If we need to define a combination of elements and at the same time some attributes of each element, we can use an array of records. For example, in a class of 30 students, we can have an array of 30 records, each record representing a student.

Figure 12.8 *Array of records*



Example 12.6

However, we normally use a loop to read data into an array of record. Algorithm 12.2 shows part of the pseudocode for this process.

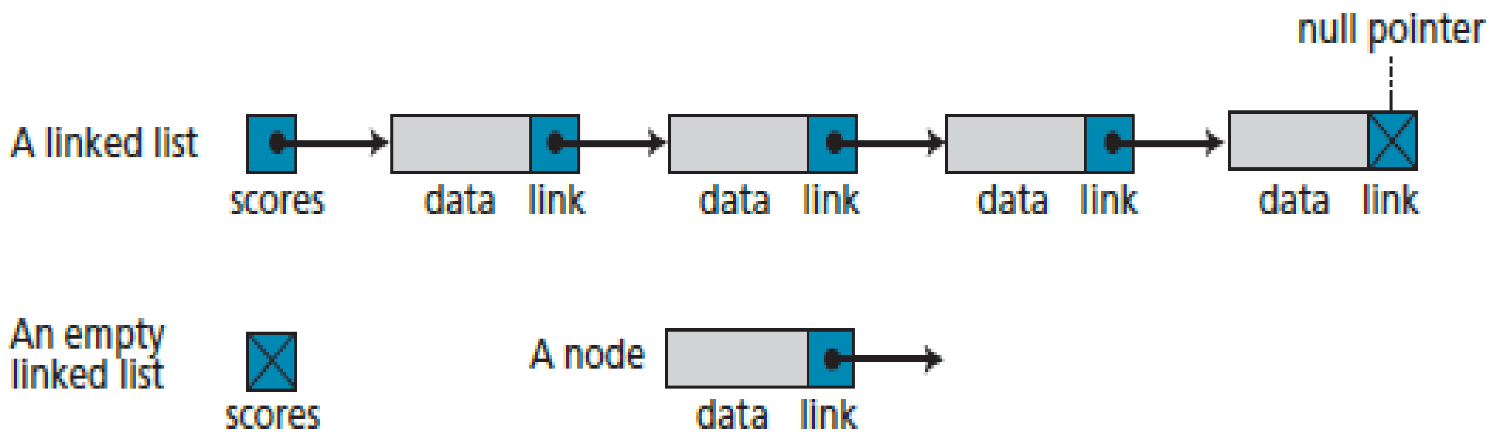
Algorithm 12.2 Part of the pseudocode to read student record

```
i ← 1
while (i < 31)
{
    read (students [i]).id
    read (students [i]).name
    read (students [i]).grade
    i ← i + 1
}
```

12.3 LINKED LISTS

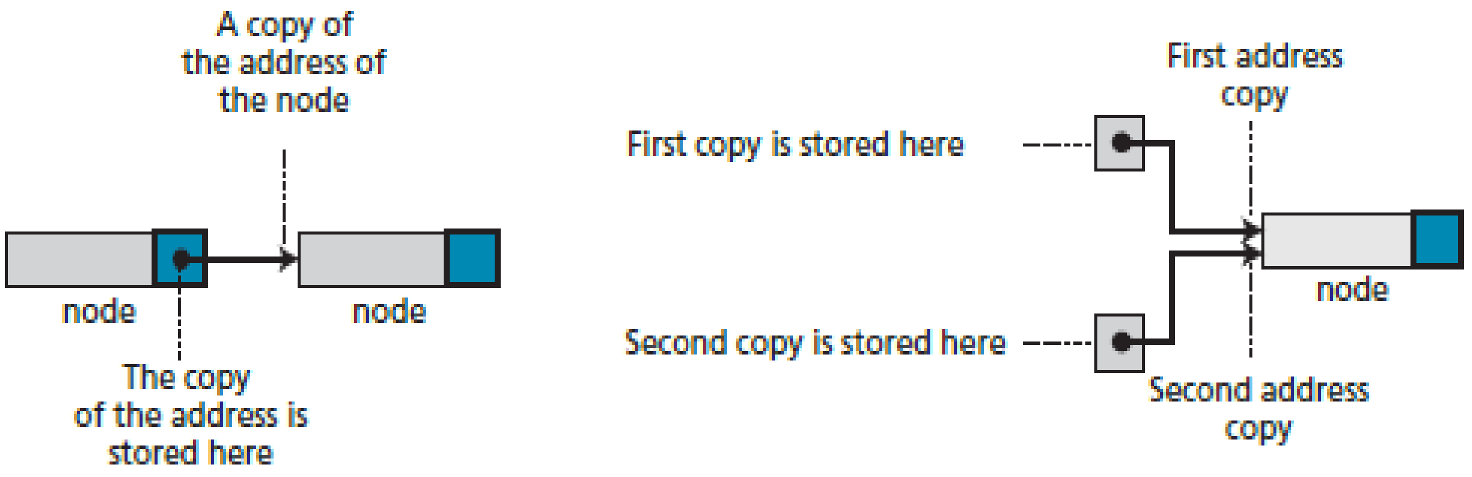
- A linked list is a collection of data in which each element contains the location of the next element—that is, each element contains two parts: data and link. The name of the list is the same as the name of this pointer variable. Figure 12.9 shows a linked list called *scores* that contains four elements. We define an empty linked list to be only a null pointer: Figure 12.9 also shows an example of an empty linked list.

Figure 12.9 *Linked lists*



- Before further discussion of linked lists, we need to explain the notation we use in the figures. We show the connection between two nodes using a line. One end of the line has an arrowhead, the other end has a solid circle.

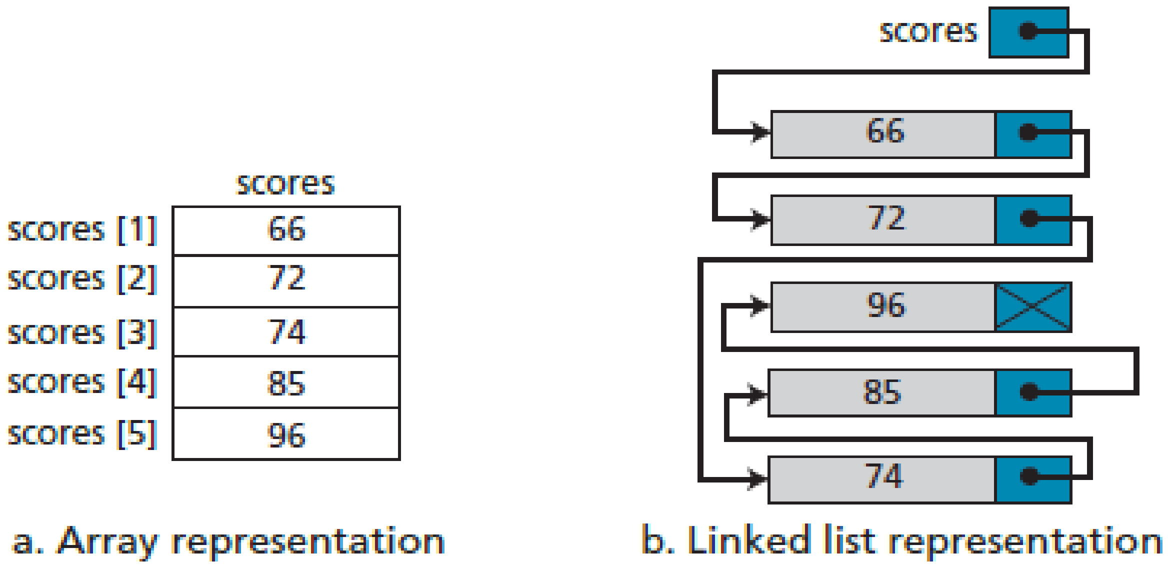
Figure 12.10 *The concept of copying and storing pointers*



Arrays versus linked lists

- Both an array and a linked list are representations of a list of items in memory. The only difference is the way in which the items are linked together. Figure 12.11 compares the two representations for a list of five integers.

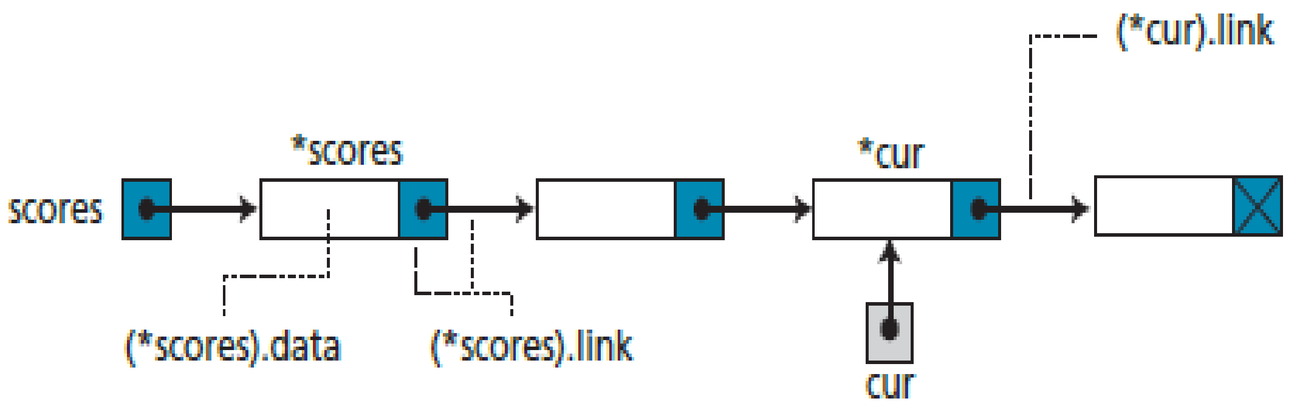
Figure 12.11 *Array* versus *linked list*



Linked list names versus nodes names

- As for arrays and records, we need to distinguish between the name of the linked list and the names of the nodes, the elements of a linked list. A linked list must have a name. The name of a linked list is the name of the head pointer that points to the first node of the list. Nodes, on the other hand, do not have an explicit names in a linked list, just implicit ones.

Figure 12.12 *The name of a linked list versus the names of nodes*



Operations on linked lists

The same operations we defined for an array can be applied to a linked list.

Searching a linked list

- Since nodes in a linked list have no names, we use two pointers, **pre** (for previous) and **cur** (for current). At the beginning of the search, the pre pointer is null and the cur pointer points to the first node. The search algorithm moves the two pointers together towards the end of the list. Figure 12.13 shows the movement of these two pointers through the list in an extreme case scenario: when the target value is larger than any value in the list.

Figure 12.13 *Moving of pre and cur pointers in searching a linked list*

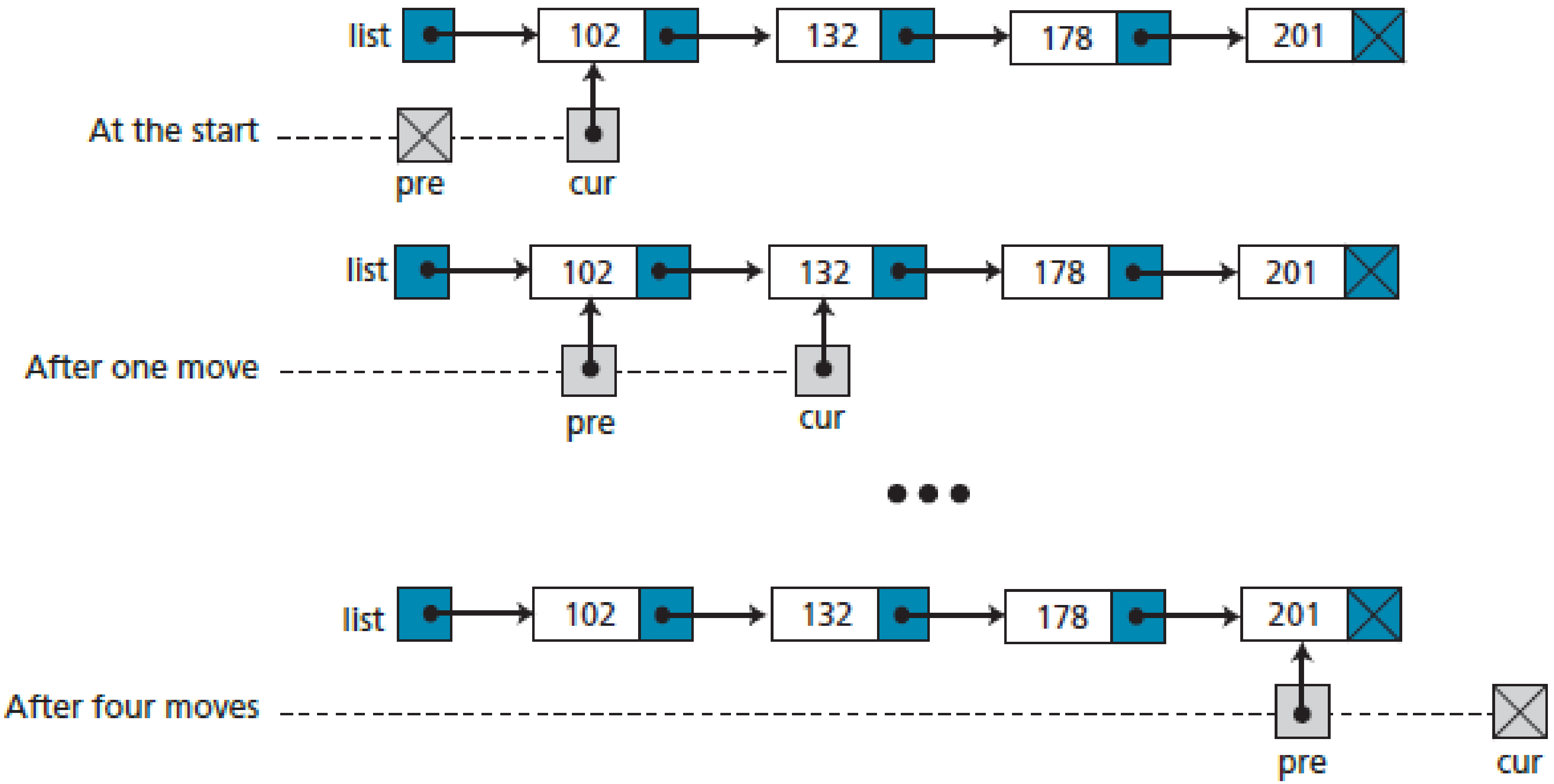
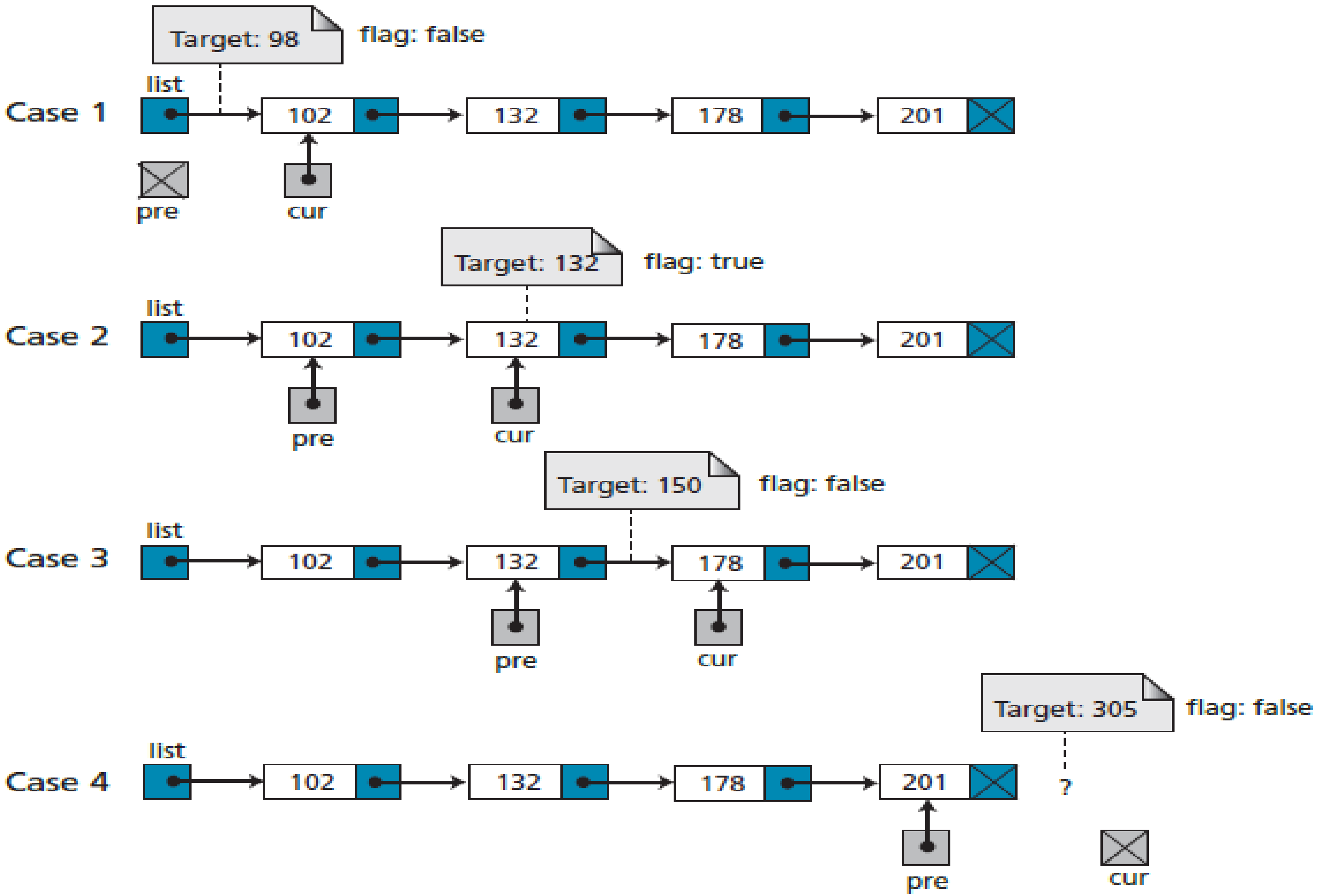


Figure 12.14 *Values of pre and cur pointers in different cases*



Algorithm 12.3 Searching a linked list

```
Algorithm: SearchLinkedList
Purpose: Search the list using two pointers: pre and cur.
Pre: The linked list (head pointer) and target value
Post: None
Return: The position of pre and cur pointers and the value of the flag
{
    pre ← null
    cur ← list
    while (target < (*cur).data)
    {
        pre ← cur
        cur ← (*cur).link
    }
    if ((*cur).data = target)   flag ← true
    else   flag ← false
}
```

Inserting a node

Before insertion into a linked list, we first apply the searching algorithm. If the flag returned from the searching algorithm is false, we will allow insertion, otherwise we abort the insertion algorithm, because we do not allow data with duplicate values.

Four cases can arise:

- Inserting into an empty list.
- Insertion at the beginning of the list.
- Insertion at the end of the list.
- Insertion in the middle of the list.

Figure 12.15 *Inserting a node at the beginning of a linked list*

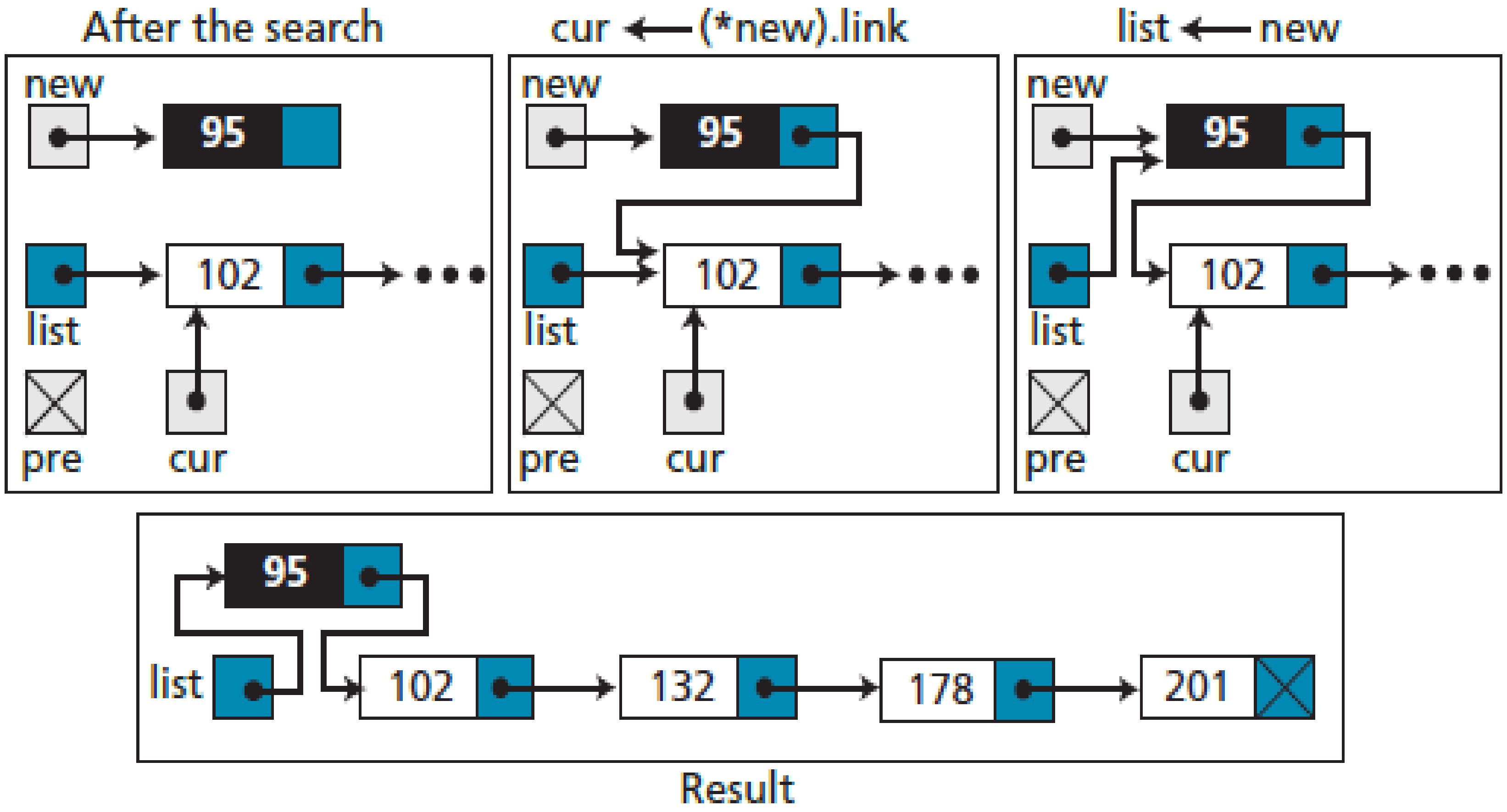


Figure 12.16 *Inserting a node at the end of the linked list*

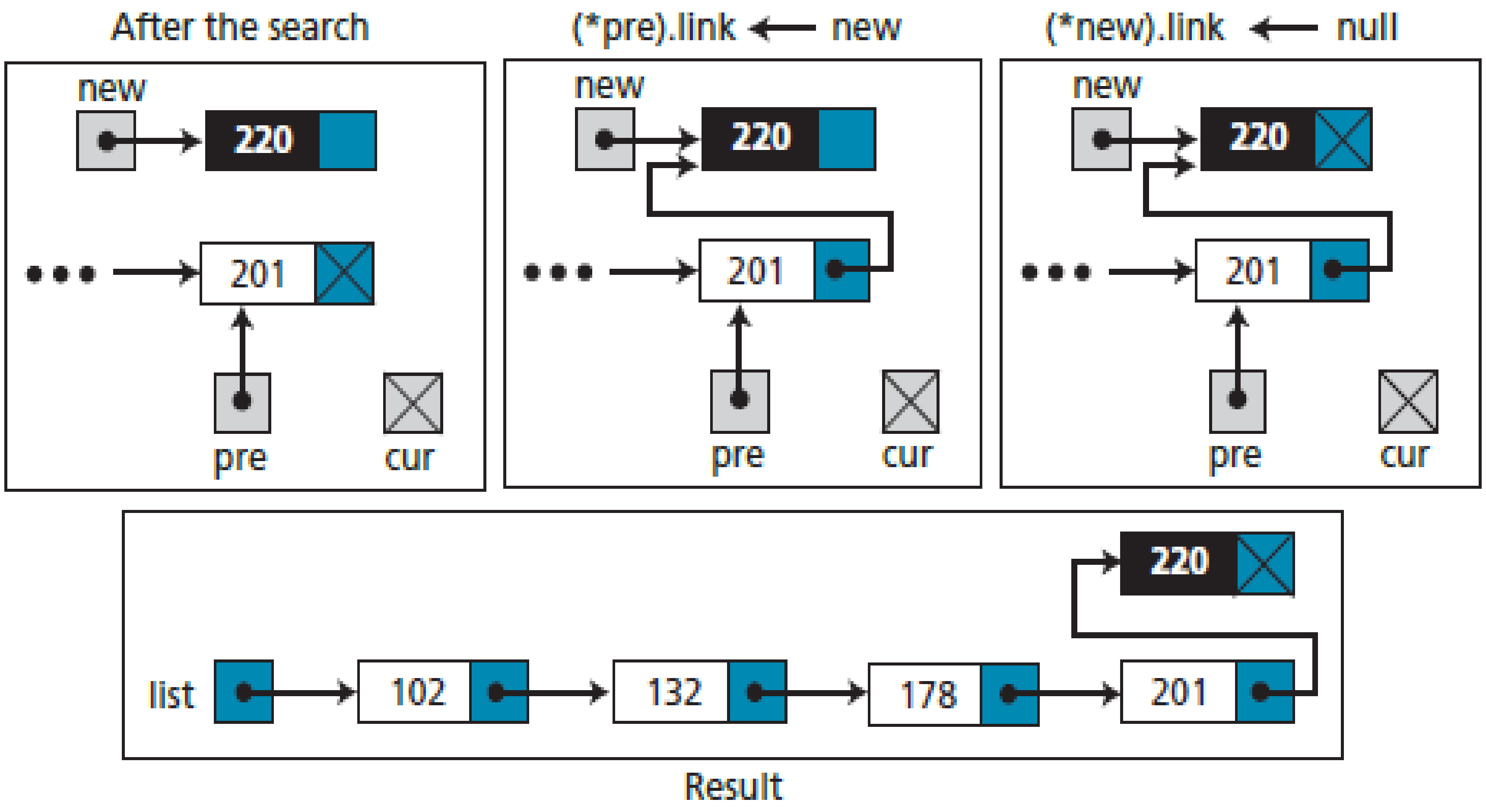
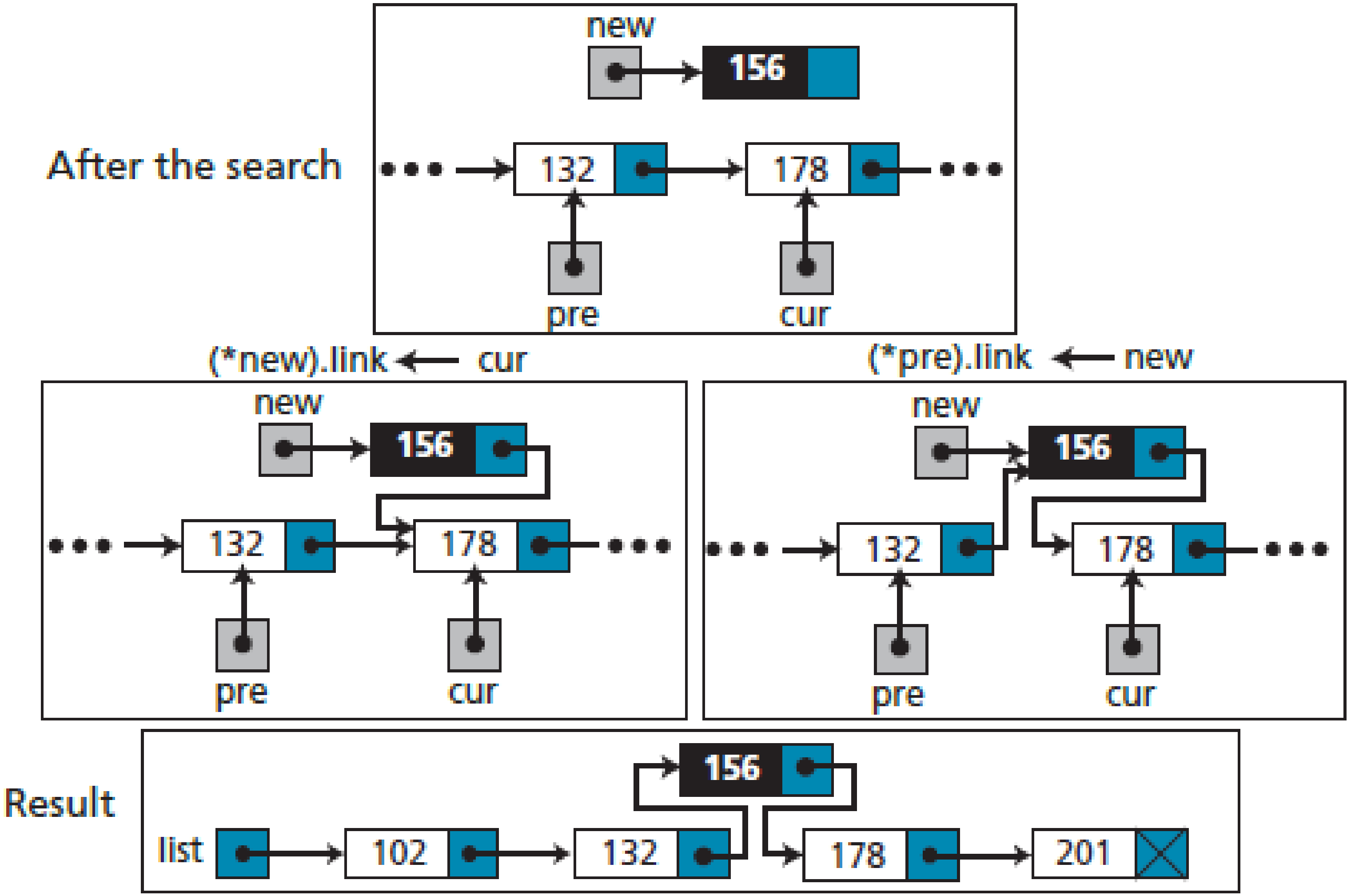


Figure 12.17 *Inserting a node at the middle of the linked list*



Algorithm 12.4 Inserting a node in a linked list

```
Algorithm: InsertLinkedList (list, target, new)
Purpose: Insert a node in the link list after searching the list
Pre: The linked list and the target data to be inserted
Post: None
Return: The new linked list
{
    searchlinkedlist (list, target, pre, cur, flag)
    // Given target and returning pre, cur, and flag
    if (flag = true)   // No duplicate
    {
```

Algorithm 12.4 Inserting a node in a linked list (continued)

```
        return list
    }
    if (list = null     // Insert into empty list
    {
        list ← new
    }
    if (pre = null)     // Insertion at the beginning
    {
        (*new).link ← cur
        list ← new
        return list
    }
    if (cur = null)     // Insertion at the end
    {
        (*pre).link ← new
        (*new).link ← null
        return list
    }
        (*new).link ← cur   // Insertion in the middle
        (*pre).link ← new
        return list
}
```

Deleting a node

- Before deleting a node in a linked list, we apply the search algorithm. If the flag returned from the search algorithm is true (the node is found), we can delete the node from the linked list. However, deletion is simpler than insertion: we have only two cases—deleting the first node and deleting any other node. In other words, the deletion of the last and the middle nodes can be done by the same process.

Figure 12.18 *Deleting the first node of a linked list*

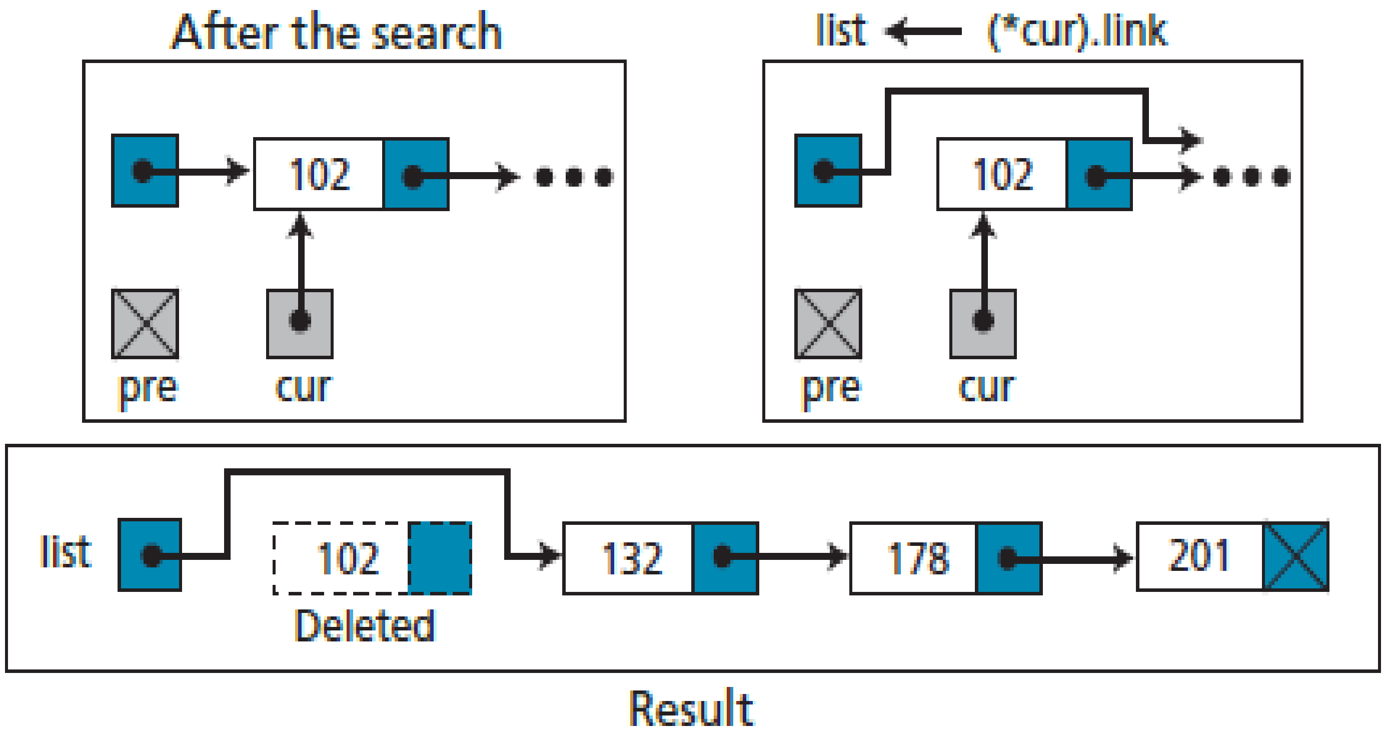
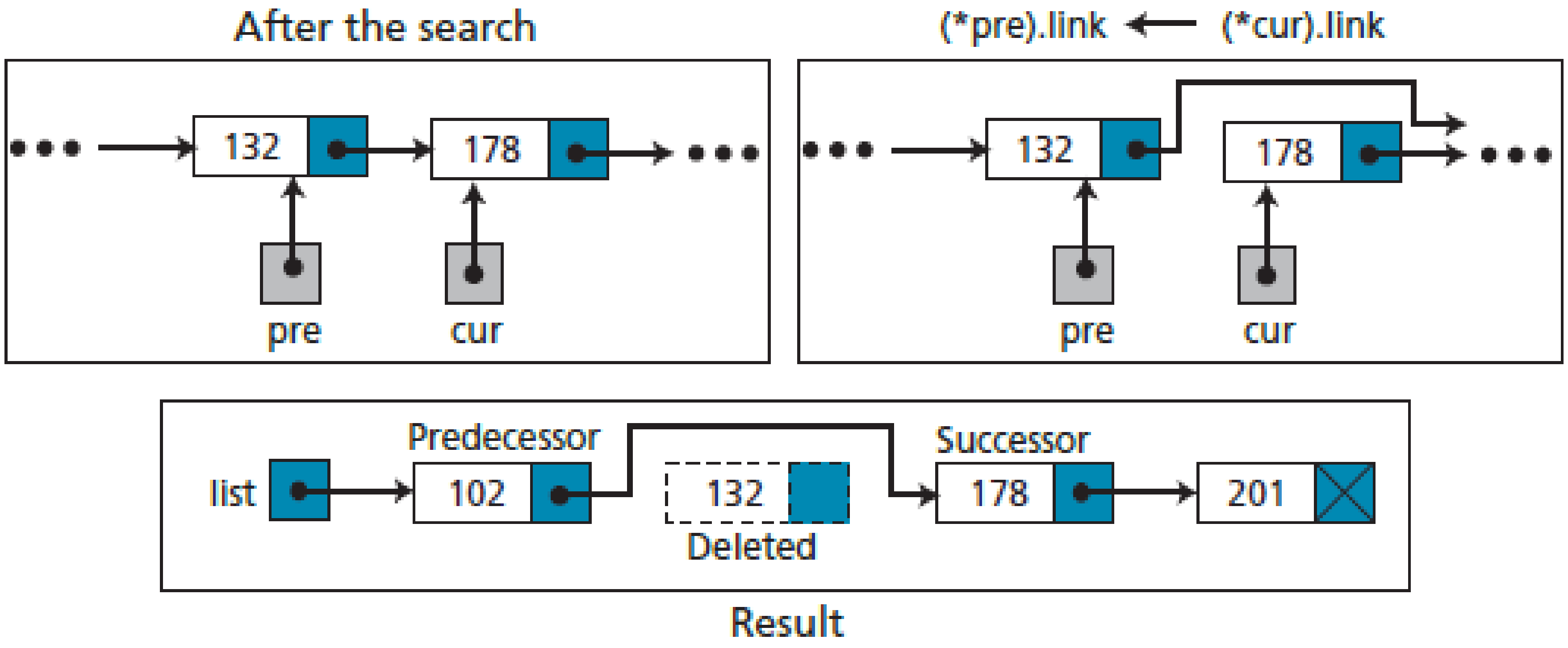


Figure 12.19 *Deleting a node at the middle or end of a linked list*



Algorithm 12.5 Deleting a node in a linked list

```
Algorithm: DeleteLinkedList (list, target)
Purpose: Delete a node in a linked list after searching the list for the right node
Pre: The linked list and the target data to be deleted
Post: None
Return: The new linked list

{
     // Given target and returning pre, cur, and flag
    searchlinkedlist (list, target, pre, cur, flag)
    if (flag = false)
    {
    return list        // The node to be deleted not found
    }
    if (pre = null)         // Deleting the first node
    {
        list ← (*cur).link
        return list
    }
    (*pre).link ← (*cur).link   // Deleting other nodes
    return list
}
```

Retrieving a node

- Retrieving means randomly accessing a node for the purpose of inspecting or copying the data contained in the node. Before retrieving, the linked list needs to be searched. If the data item is found, it is retrieved, otherwise the process is aborted. Retrieving uses only the cur pointer, which points to the node found by the search algorithm. Algorithm 11.6 shows the pseudocode for retrieving the data in a node. The algorithm is much simpler than the insertion or deletion algorithm.

Algorithm 12.6 Retrieving a node in a linked list

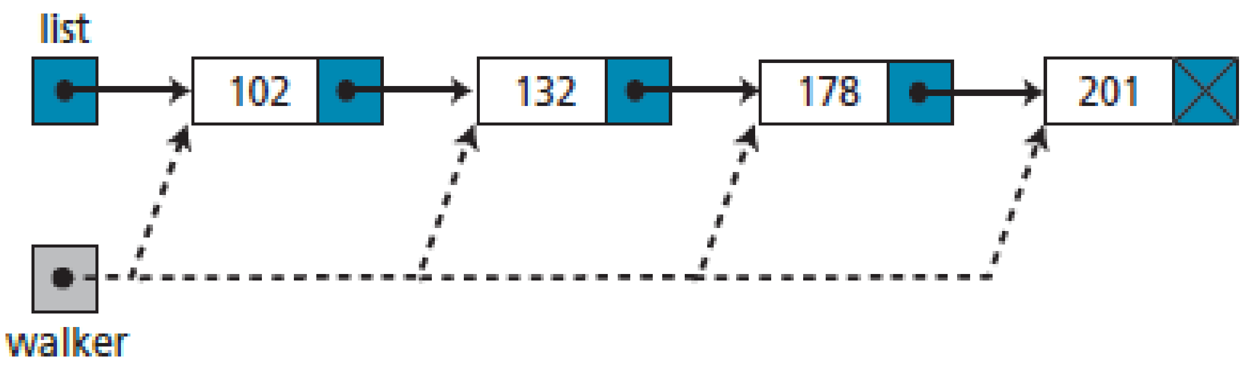
```
Algorithm: RetrieveLinkedList (list, target)
Purpose: Retrieves the data in a node after searching the list for the right node
Pre: The linked list (head pointer) and the target (data to be retrieved)
Post: None
Return: Return the data retrieved

{
    searchlinkedlist (list, target, pre, cur, flag)
    if (flag = false)     // The node not found
    {
        return error
    }
    return (*cur).data
}
```

Traversing a linked list

- To traverse the list, we need a "walking" pointer, which is a pointer that moves from node to node as each element is processed. We start traversing by setting the walking pointer to the first node in the list. Then, using a loop, we continue until all of the data has been processed. Each iteration of the loop processes the current node, then advances the walking pointer to the next node. When the last node has been processed, the walking pointer becomes null and the loop terminates (Figure 12.20).

Figure 12.20 *Traversing a linked list*



Algorithm 12.7 Traversing a linked list

```
Algorithm: TraverseLinkedList (list)
Purpose: Traverse a linked list and process each data item
Pre: The linked list (head pointer)
Post: None
Return: The list

{
    walker ← list
    while (walker ≠ null)
    {
        Process (*walker).data
        walker ← (*walker).link
    }
    return list
}
```

Applications of linked lists

- A linked list is a very efficient data structure for sorted list that will go through many insertions and deletions. A linked list is a dynamic data structure in which the list can start with no nodes and then grow as new nodes are needed. A node can be easily deleted without moving other nodes, as would be the case with an array. For example, a linked list could be used to hold the records of students in a school. Each quarter or semester, new students enroll in the school and some students leave or graduate.

A linked list is a suitable structure if a large number of insertions and deletions are needed, but searching a linked list is slower that searching an array.

Exercises

- P.320 Problems
  - p11-2, p11-5, p11-13, p11-18, p11-19.